Fine-Tuning Large Language Models Using LoRA

Daryna Antoniuk Olha Kaplysh Maryna Ohinska

Ukrainian Catholic University
Faculty of Applied Sciences

https://github.com/darynaantoniuk/LoRA_LA_project

Aim, Scope, and Rubric Coverage

Aim

The project studies whether **low-rank matrix decomposition** can replace full dense fine-tuning updates in `roberta-base`, while preserving downstream performance and reducing trainable parameters by an order of magnitude.

Research question

Can task-specific adaptation be restricted to a low-dimensional subspace without sacrificing practical utility on GLUE classification tasks?

Why LoRA

Problem with full fine-tuning

- all pretrained weights are updated,
- optimizer states are stored for all parameters,
- a separate dense checkpoint is needed for each task.

Why LoRA

- freeze the pretrained operator,
- train only a structured low-rank correction,
- merge the correction back after training.

Method choice

Method	Main drawback
Full fine-tuning	Maximum training cost
Prompt tuning	Weaker internal control
Adapters	Inference overhead
LoRA	Best linear-algebra trade-off

Repository setting

Base model: `roberta-base`
Tasks in code: SST-2, MRPC, CoLA, MNLI

LoRA as a Rank-Constrained Update

For a pretrained linear map $W \in \mathbb{R}^{k \times d}$,

$$h = Wx.$$

LoRA replaces the unrestricted update by

$$W' = W + \Delta W, \qquad \Delta W = BA, \qquad \text{rank}(\Delta W) \leq r,$$

with

$$A \in \mathbb{R}^{r \times d}, \qquad B \in \mathbb{R}^{k \times r}, \qquad r \ll \min(k, d).$$

Forward pass in the implementation:

$$h = Wx + \frac{\alpha}{r} B(Ax).$$

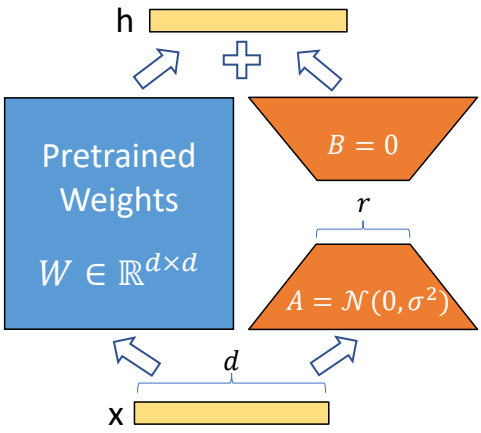


Figure: LoRA reparametrization. The pretrained weight matrix W is frozen; only A and B are trained.

SVD and Truncated SVD

Core formulas

$$W = U\Sigma V^\top, \qquad W_k = U_k \Sigma_k V_k^\top, \qquad W_k = \arg \min_{\text{rank}(X) \le k} \|W - X\|_F.$$

Connection to eigenvectors

$$W^\top W \, v_i = \sigma_i^2 v_i.$$

Therefore, dominant right singular directions can be extracted iteratively from $W^\top W$.

Why this matters for LoRA

If the useful adaptation directions are concentrated in a small singular subspace (it has low intrinsic rank), then a rank-r update is not only parameter-efficient but also mathematically justified by best low-rank approximation theory (Eckart-Young-Mirsky theorem).

Custom LoRA Algorithm

Truncated SVD by power iteration and deflation

1. Set residual matrix $R \leftarrow W$.
2. For each component, iterate

$$x_{t+1} = \frac{Mx_t}{\|Mx_t\|}, \qquad M = R^\top R,$$

   until convergence to a dominant right singular direction.
3. Recover

$$\sigma = \|Rv\|, \qquad u = \frac{Rv}{\sigma}.$$

4. Deflate the residual:

$$R \leftarrow R - \sigma uv^\top.$$

How the code uses it

- `truncated_svd_power_iteration()`
- `choose_truncated_rank()`
- `LoRALinear.init_with_truncated_svd()`

Implementation role

We use truncated SVD for **structured LoRA initialization**.

Implementation Overview

File	Verified role
`src/lora.py`	LoRA layer, custom truncated SVD, freezing, merging, parameter counting
`src/train.py`	Model loading, LoRA injection, Hugging Face Trainer setup
`src/evaluate.py`	Checkpoint evaluation, throughput, implicit singular-value estimation
`src/data.py`	GLUE loading, task-specific tokenization, split handling
`src/config.py` `configs/train.yaml`	Rank, learning rate, batch size, mode, seed
`scripts/run_finetuning.py`	Rank and task sweeps
`scripts/run_lora_modes_benchmarks.sh`	MRPC/CoLA benchmark launcher

Layer injection logic

The code scans RoBERTa for `nn.Linear` modules whose names contain `query` or `value`, replaces them with `LoRALinear`, and freezes all non-LoRA parameters.

Data, Experimental Setup, and Evaluation Protocol

Datasets in code

- MRPC: paraphrase detection
- CoLA: linguistic acceptability

Preprocessing

RoBERTa tokenizer, truncation

Default configuration

- model: `roberta-base`
- rank: 8
- $\alpha = 16$
- learning rate: $2 \cdot 10^{-4}$
- batch size: 16
- epochs: 3
- seed: 42

Metrics

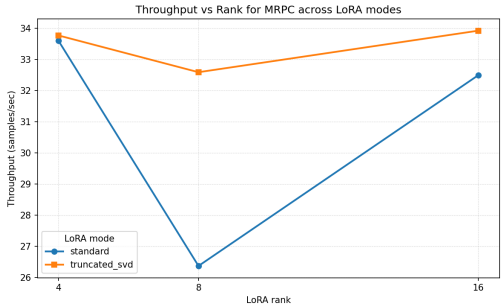
accuracy, F1, trainable parameter count, throughput, GPU memory when CUDA is available.

Results

Task	Mode	r	Acc.	F1
CoLA	Full FT	–	69.1%	81.7%
CoLA	LoRA std	4	80.6%	86.9%
CoLA	LoRA TSVD	4	81.0%	87.3%
MRPC	Full FT	–	68.4%	81.2%
MRPC	LoRA std	16	84.8%	89.0%
MRPC	LoRA TSVD	16	86.0%	89.8%

Representative comparison

Best LoRA checkpoints outperform the stored full fine-tuning baselines on both tasks.



MRPC throughput vs rank across LoRA modes

Conclusion

- LoRA consistently beats the stored full fine-tuning baselines on CoLA and MRPC.
- The best observed runs are **truncated-SVD LoRA**: CoLA at $r = 4$ and MRPC at $r = 16$.
- The gain is achieved with only 147,456–589,824 trainable parameters instead of 124.6M.
- Increasing rank does not guarantee better results; task-dependent low rank is sufficient.